

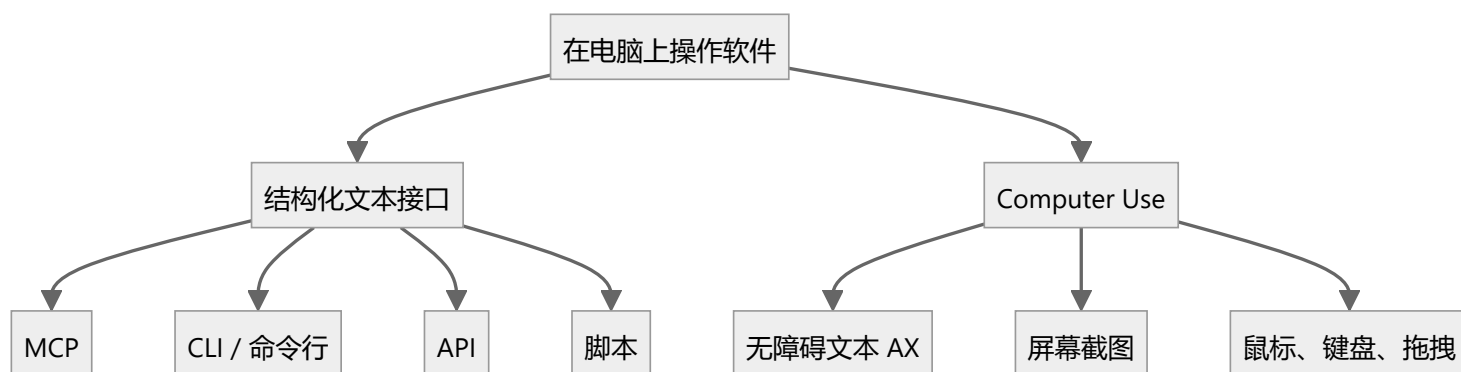
《Computer Use 是什么？它真的有在看你的屏幕吗？》整理

先说结论：视频想纠正一个很常见的直觉。

很多人以为 Computer Use 就是：模型像人一样盯着屏幕，找到按钮，移动鼠标，点下去，再输入键盘。但在 GPT-6 和 Codex 这类工具里，真实情况往往相反：它会优先用脚本、API、命令行、MCP 这类文本接口完成任务。只有这些方式不方便时，才退回“看界面、点鼠标”的 GUI 操作。即使退回 GUI，它也不是只靠截图看图，而是先读取一种叫“无障碍文本”的结构化文字描述。

一、先建立知识地图

可以这样理解“在电脑上操作软件”的几种方式：



这里的重点不是谁高级谁低级，而是优先级。

对模型来说，文本接口通常更稳定、更省 token、更容易一次做对。GUI 操作更像备用方案。

二、Blender 实验：看起来在操作软件，其实在写脚本

作者打开 Codex，切到 Agent 模式，再打开 Blender，让模型“在 Blender 里画个球”。中间还遇到 macOS 系统版本过低的问题，升级之后继续。

结果不是鼠标在屏幕上点点点，而是模型打开了 Blender 的脚本控制台，开始写脚本。

我会先查看 Blender 和当前工程，再搭建完整别墅场景，配置室内家具、泳池景观、材质与金色时刻的运镜，并保存工程。

🔥 已使用 Org.blenderfoundation.blender 集成

场景会采用暖白石材、深色金属与木饰面的两层现代别墅：一层客餐厨贯通，二层卧室连接露台，前庭配置无边泳池、躺椅与地中海式绿植。镜头将从泳池前方缓慢环绕到别墅侧面。

🔥 已使用 Org.blenderfoundation.blender 集成 ▾

🔥 打开 Blender 场景脚本控制台

🔥 搭建别墅、家具、泳池、景观和15秒运镜

🔥 检查脚本输入状态

🔥 切换编辑器类型

🔥 进入 Python 控制台

🔥 运行完整场景创建脚本

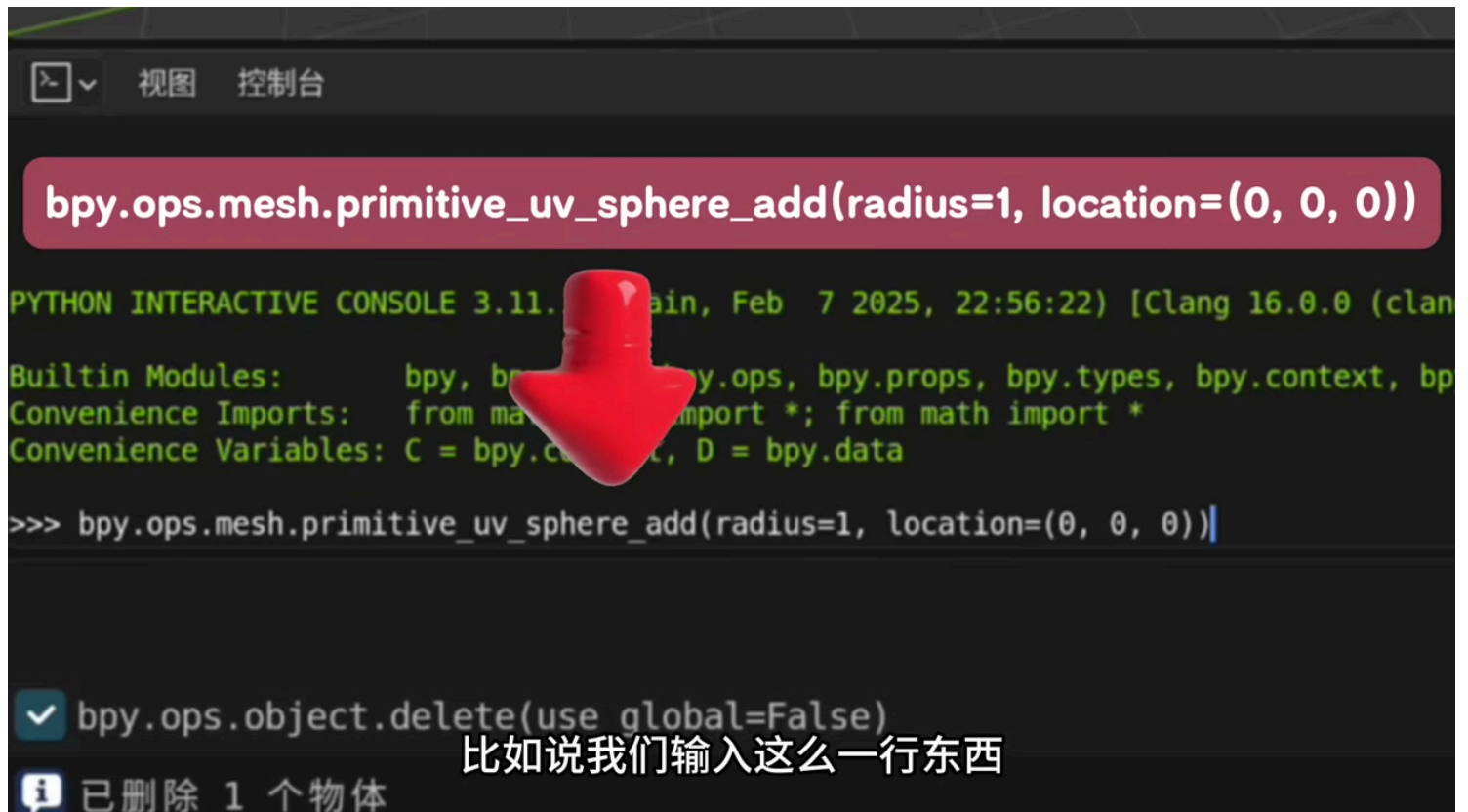
🔥 检查 Blender 控制台

🔥 执行场景生成并保存工程

我正在处理中文版 Blender 的材质与脚本。脚本已经准备好，接下来会检查实际渲染、调整构图与可见细节。

就应该开始点点点了吧

Blender 的脚本控制台可以理解成一个输入代码的地方，格式类似 Python。输入一行代码，它就能在画面中心画出一个球。



```
> ▾ 视图 控制台
```

```
bpy.ops.mesh.primitive_uv_sphere_add(radius=1, location=(0, 0, 0))
```

```
PYTHON INTERACTIVE CONSOLE 3.11. (main, Feb 7 2025, 22:56:22) [Clang 16.0.0 (clang-1600.0.2)]
```

```
Builtin Modules:      bpy, bpy.ops, bpy.props, bpy.types, bpy.context, bpy.data
```

```
Convenience Imports:  from math import *
```

```
Convenience Variables: C = bpy.context, D = bpy.data
```

```
>>> bpy.ops.mesh.primitive_uv_sphere_add(radius=1, location=(0, 0, 0))
```

```
✓ bpy.ops.object.delete(use_global=False)
```

```
已删除 1 个物体
```

比如说我们输入这么一行东西

后面那个“永世别墅”的例子更明显。模型不是靠鼠标一个个摆家具，而是用大量脚本创建了几百个对象，最后包含 650 个场景对象。整个过程中，唯一比较像 GUI 操作的动作，可能只是点到脚本控制台里面。

所以这里的关键是：

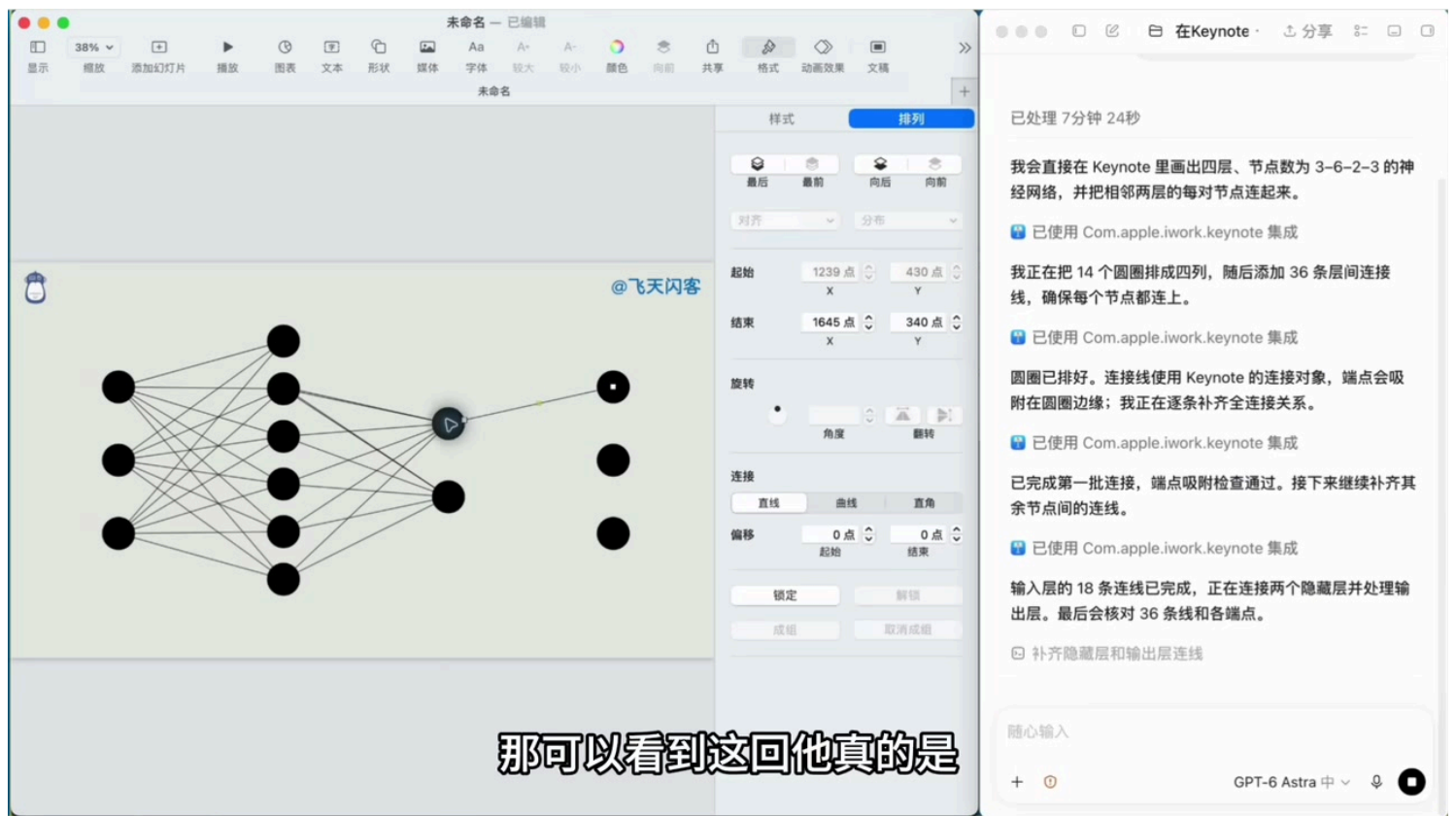
表面上它在“操作 Blender”，实际上它主要在写代码。

对人类来说，用脚本一行一行建别墅很反常；但对模型来说，这反而更自然。因为模型本来就更擅长处理文本和代码，而不是移动鼠标、精确点击。

三、Keynote 实验：这次真的点鼠标了，但背后还是文本

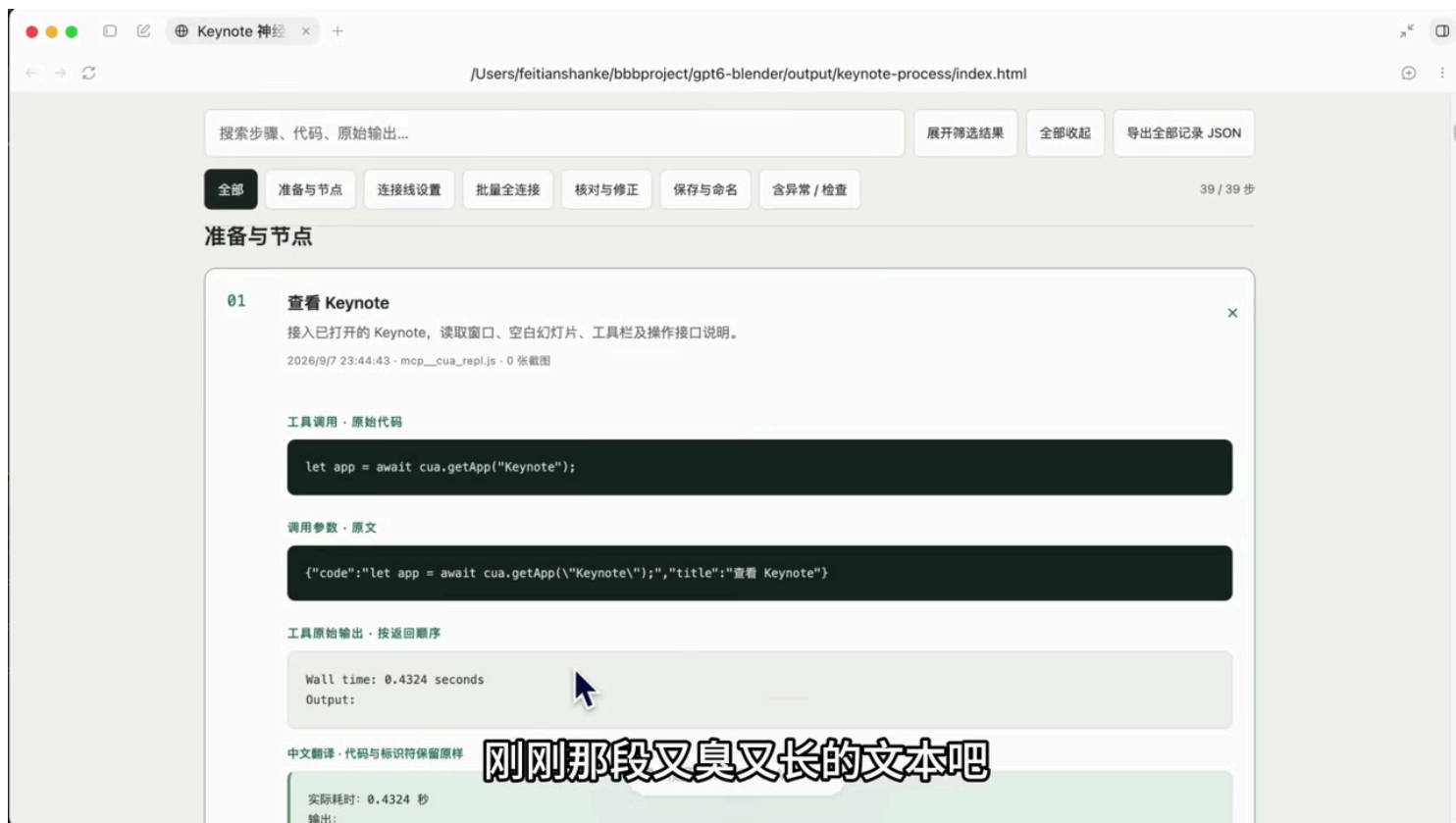
为了看到“模型像人一样操作界面”，作者换了一个更简单的软件：Keynote，也就是 Windows 上的 PPT。任务是在里面画一个神经网络。

这次画面看起来确实符合直觉：模型移动鼠标，一笔一笔画图，中间还画错了两根线，最后自查并纠正。

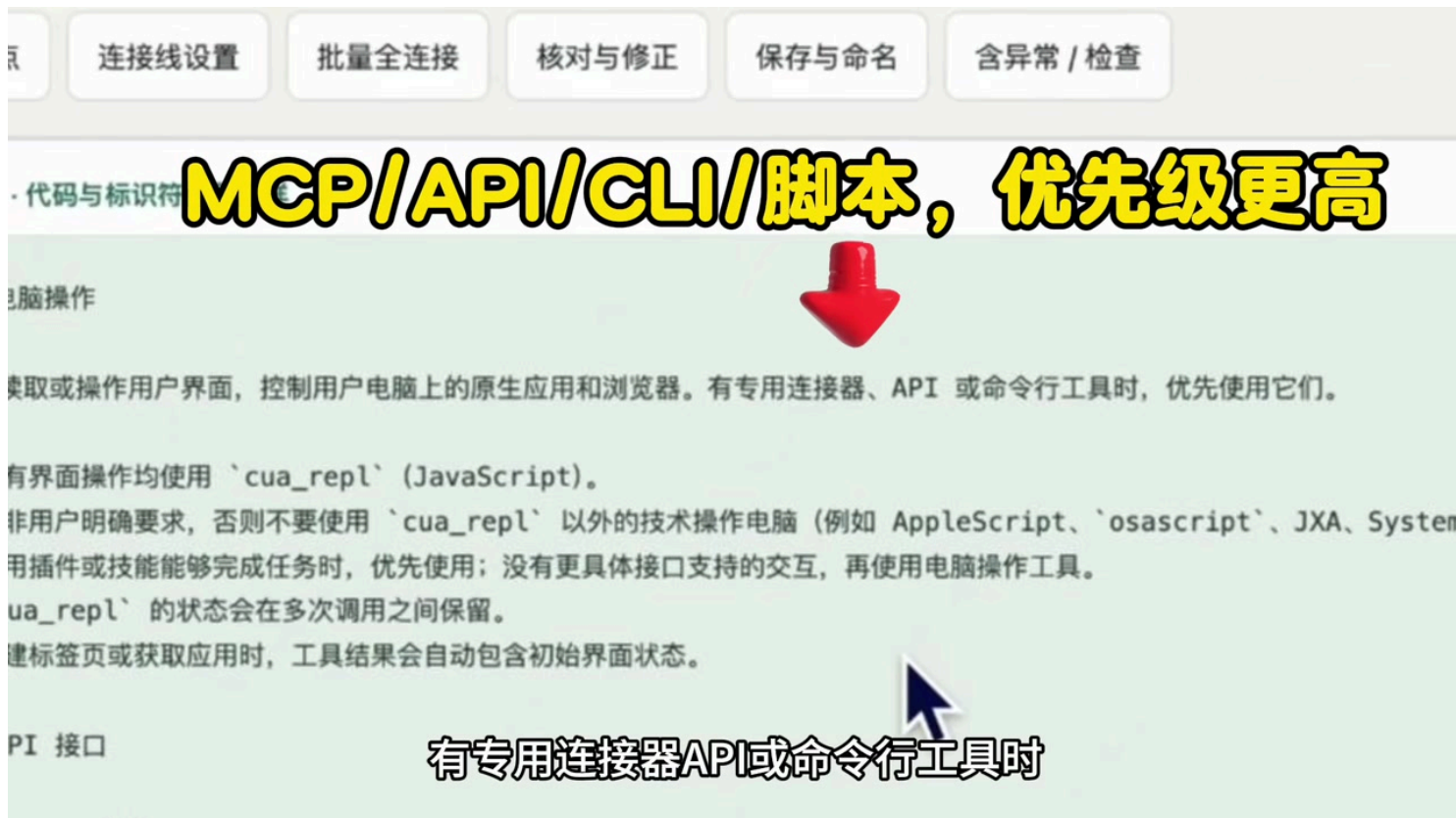


但如果继续追查它每一步到底做了什么，会发现事情并不简单。

第一步不是“看屏幕找按钮”，而是调用了一个叫 `getapp` 的方法，传入名字 `keynote`，然后返回一大段文本。这段文本描述的是 Keynote 当前的状态信息。



更重要的是 Computer Use 的工具说明里有一句话：
如果有专用连接器、API 或命令行工具，优先使用它们。



这句话说明，GUI 操作不是第一选择，而是排在连接器、API、命令行、脚本之后的备用方案。
这也解释了为什么在 Blender 里，模型会优先写脚本，而不是用鼠标建模。

四、即使必须操作界面，它也不是直接“看图”

工具说明里还有一句很关键：执行一个或多个界面动作后，在决定下一步之前，要调用 `getAXState` 方法，掌握当前页面状态，并从最新的无障碍文本中重新获取元素信息，避免沿用过期信息。

`getAXState` 返回的是“无障碍文本”。

准备与节点

连接线设置

批量全连接

保存命令

含异常 /

getAXState()

无障碍文本

```
## 工作流程

执行一个或多个界面操作后，在决定下一步之前调用 `getAXState()`。这样可掌握当前界面状态，并从最新的无障碍文本中重新获取元素索引，避免沿用过期信息。为节省令牌，适当时会以相对于上一次无障碍树的差异形式返回结果，仅列出删除、新增或更改的元素。优先使用默认差异输出；只有需要最新的完整无障碍树时，才使用 `{ disableDiffing: true }`。如果上一次只查看了截图，在再次依赖无障碍索引前，应请求完整树。

在保持界面状态最新的同时，减少模型与工具之间的往返调用：

- 将确定性操作和随后的 `getAXState()` 合并为一次调用。可在同一次调用中操作界面并返回更新状态，无须另发一次工具调用。
- 调用 `cua.getApp(...)`、`cua.getTab(...)` 和 `cua.createBrowserTab(...)` 会返回应用或标签页绑定，并自动显示运行后的最新状态。
- 如果单独调用 `getAXState()` 报告无障碍树没有变化，在没有其他操作的情况下不要立即重复调用。只有明确知道缺少哪些上下文、且相应表示方式才使用 `getScreenshot()`、`getAXStateAndScreenshot()` 或 `{ disableDiffing: true }`。
- 优先使用当前状态中已经可见且直接相关的结果，而不是打开“显示全部”等范围更广的中间界面。
- 所请求的结果已经可见时，停止探索并答复。

执行一个或多个操作，然后获取最新状态：

```typescript
await target.click(42);
await target.setValue(42, "openai.com");
```

### 那么也需要先调用一个叫

无障碍文本可以理解成：把屏幕上的界面结构，用文字描述出来。比如有哪些窗口、按钮、输入框、菜单、元素位置和状态。它不是一张图片，而是一段结构化文字。

全部

准备与节点

连接线设置

批量全连接

核对与修正

保存与命名

含异常 / 检查

39 / 39 步

Window: "未命名", App: Keynote讲演.

0 标准窗口 未命名, Secondary Actions: Raise

1 container 格式化工具

2 单选按钮 Description: 幻灯片, Value: 1

3 按钮 Description: 幻灯片布局, 标题, Help: 为所选幻灯片选取不同的布局。

4 滚动区 “幻灯片”格式化工具, Secondary Actions: Scroll Up, Scroll Down

5 文本 外观

6 复选框 标题, Help: 选择此项以添加标题文本框。 , Value: 0

7 复选框 正文, Help: 选择此项以添加正文文本框。 , Value: 0

8 复选框 幻灯片编号, Help: 选择此项以添加幻灯片编号。 , Value: 0

9 文本 背景

10 单选按钮 Description: 标准, Help: 为幻灯片添加颜色、渐变或图像填充背景。 , Value: 1

11 单选按钮 Description: 动态, Help: 为幻灯片添加动态背景。 , Value: 0

12 按钮 点按以选取预置颜色、渐变或图像填充。

13 文本 当前填充

14 弹出式按钮 Help: 选取填充类型。 , Value: 图像填充

15 图像 Description: 图像预览, Help: 将图像文件拖到这里。

16 弹出式按钮 Description: 填充技巧, Help: 选取在对象中显示图像的方式。 , Value: 缩放以

17 按钮 选取..., Help: 选取要用作填充的图像文件。

18 文本 缩放

19 步进器 (disabled) Help: 设定高分辨率图像中有多少部分可见。 , Value: 1, Secondary Actions: Increment, Decrement

20 增箭头按钮

21 减箭头按钮

22 滑块 (disabled) Help: 调整高分辨率图像中有多少部分可见。 , Value: 1, Details: 100%, Secondary Actions: Increment, Decrement

23 值指示器 1

24 文本栏 (disabled) Help: 设定高分辨率图像中有多少部分可见。 , Value: 1, Details: 100%, Secondary Actions: Increment, Decrement 不适用

25 滚动条 (disabled, settable) Help: 设定高分辨率图像中有多少部分可见。 , Value: 1, Details: 100%, Secondary Actions: Increment, Decrement

26 按钮 编辑幻灯片布局, Help: 查看、编辑和创建幻灯片布局。

27 分离组

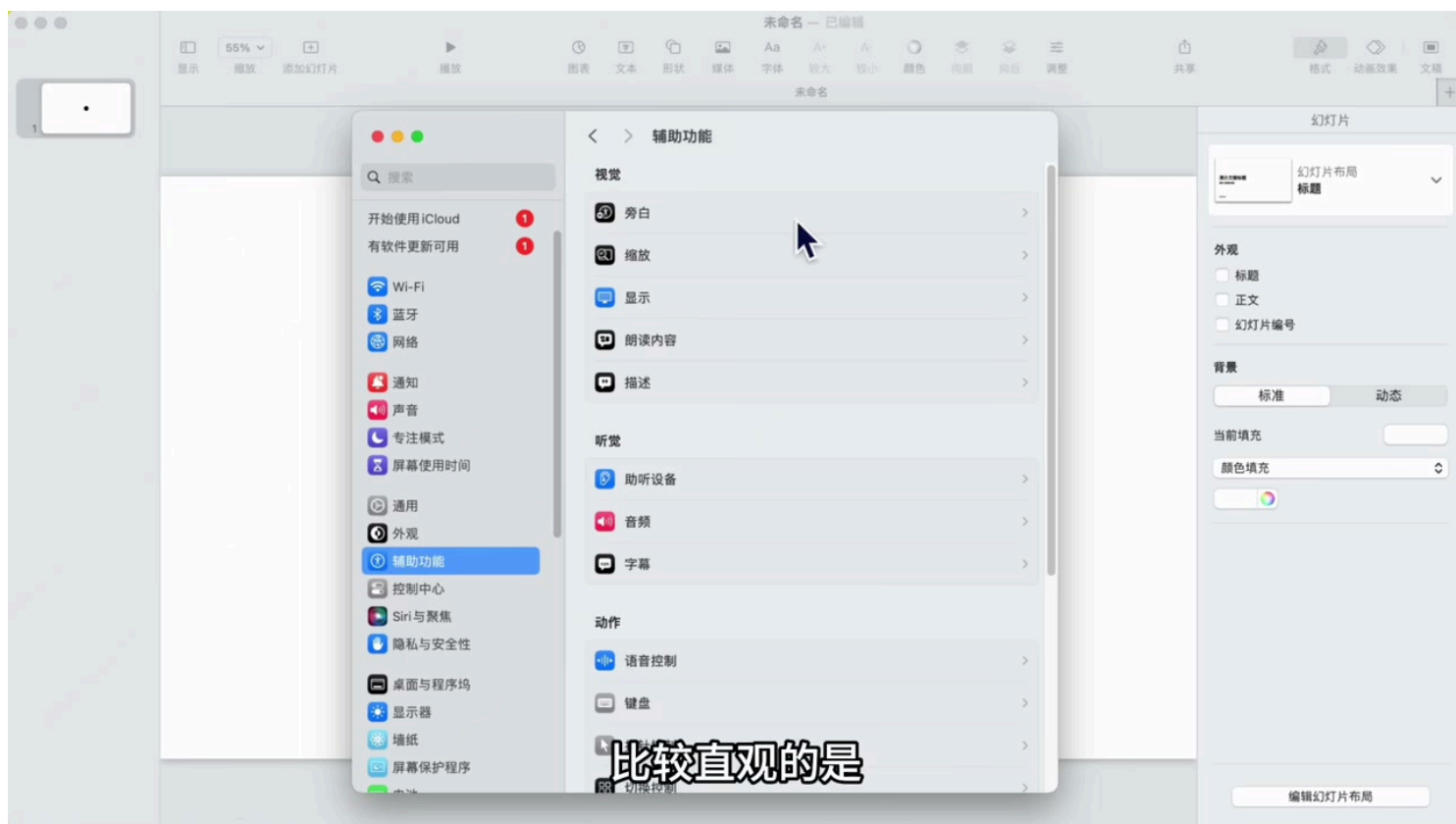
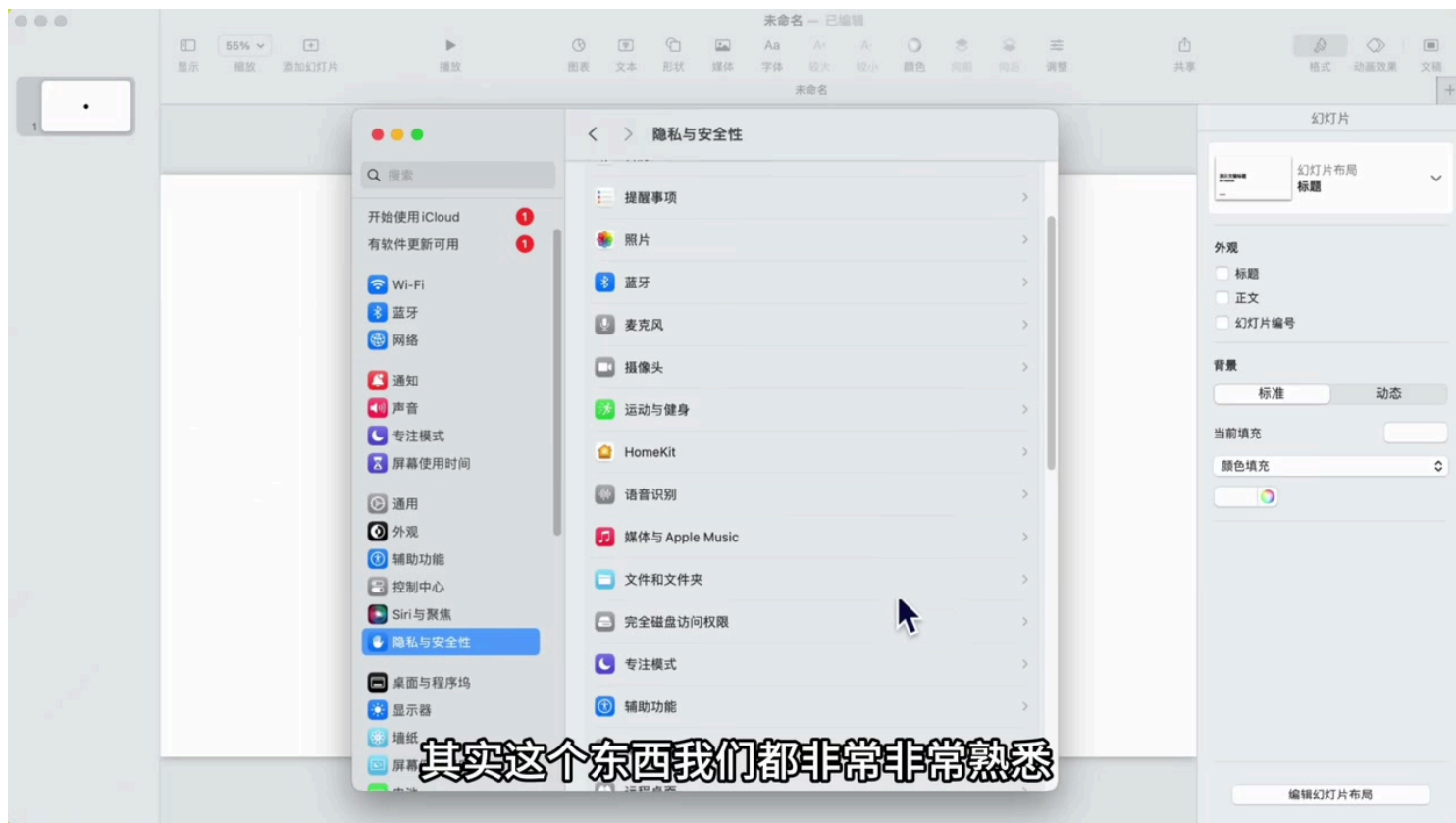
无障碍文本

第二段这个文字输出的内容

后续步骤里，模型会优先读取这种文本，而且还会用增量描述的方式，只告诉模型变化的部分，从而节省 token。

这个无障碍文本来哪里？

来自 macOS 的辅助功能权限，也就是“无障碍”功能。它本来是帮助视障人士了解屏幕内容的。比如打开旁白，电脑可以把屏幕内容读出来。



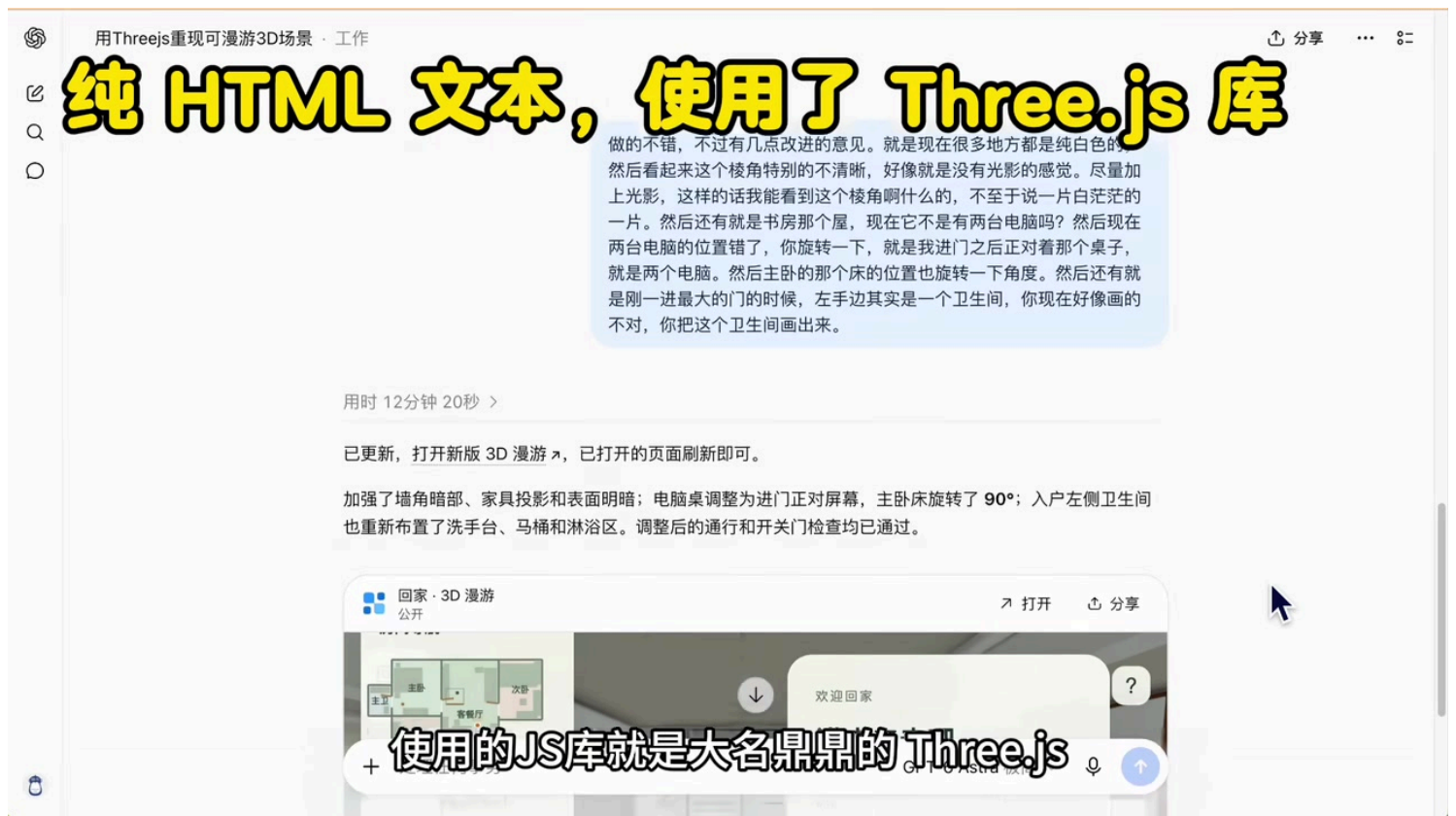
更底层，它依赖苹果官方的 AX 接口。这个接口返回类似树状结构的屏幕信息，由各个 App 自己提供，对接苹果的无障碍系统。

所以 Keynote 这个例子里，虽然模型看起来在移动鼠标、点击界面，但底层大部分信息仍然来自纯文本，而不是大模型直接看屏幕截图理解画面。

## 五、开头那个“自己家的 3D 画面”也不是 Blender

视频开头作者用自家图片生成了一个第一视角 3D 画面。这个例子容易让人以为它也是在 Blender 里操作出来的。

但实际上，它只是在 ChatGPT 网页端上传了一堆图片，然后生成了一个纯 HTML 页面，使用的是 Three.js 这个 3D 库。



最后加载和渲染速度也很快，可以直接做成第一视角游戏。



这说明网上很多看起来很酷的 3D 生成视频，不一定是用 Blender 做的。

即使是用 Blender 做的，也不一定是手动点屏幕操作的 Blender，更可能是 CLI、MCP、API 或脚本操作。

## 六、为什么这次 GPT-6 和 Blender 会火出圈？

可以归纳成几个原因：

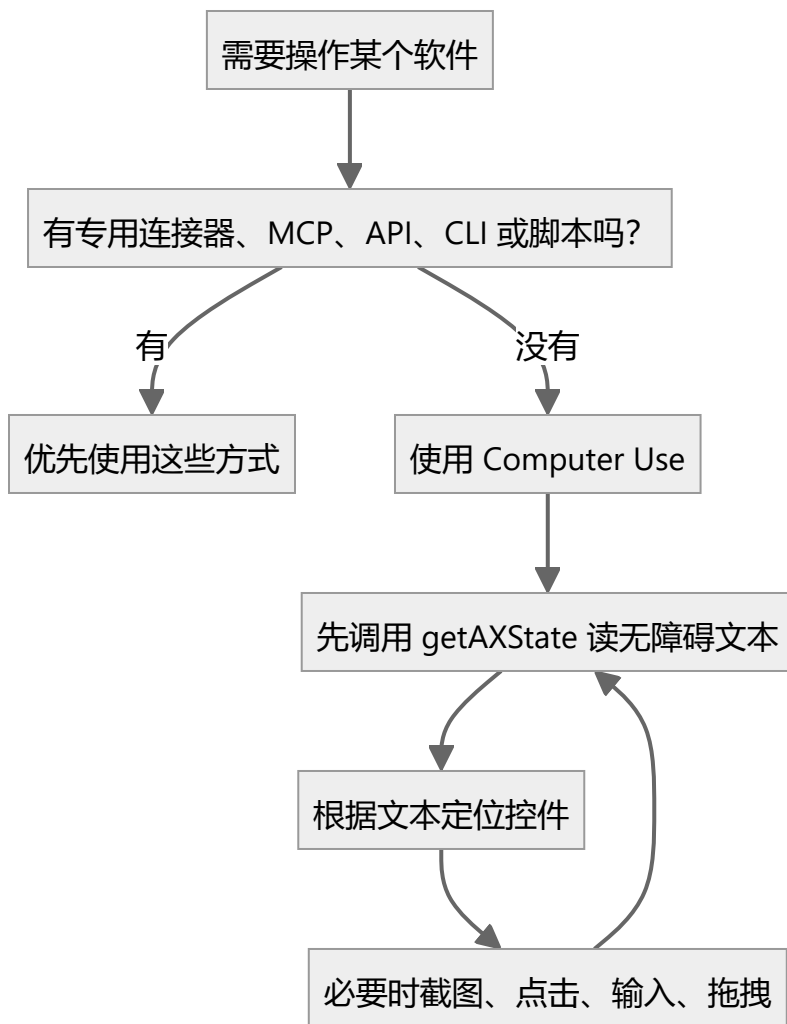
1. Computer Use 能力确实大幅提升了。作者也试过其他 Agent 产品，做不出这样的效果。
2. 它的能力刚好达到一个阈值：可以一次性、比较丝滑、没有明显卡点地完成一个三维作品创作。
3. 它的工具选择设计很合理。Computer Use 的优先级低于 MCP、CLI、API 或脚本。模型会先选自己更擅长、效率更高的方式。
4. 大模型本质上还是更擅长和文本打交道。它的处理逻辑和人不一样，效率也可能更高。

作者还提到，之前用 AppleScript 操作 Keynote 也可以画神经网络。但模型评估后，可能发现直接用 macOS 辅助功能更方便。这也是它智能的体现：不是所有操作都要硬生生去读屏幕。

## 七、一句话总结

Computer Use 的关键，不是“模拟人看屏幕、点鼠标”。

更准确的理解是：



所以，“GUI 当立，CLI、脚本、API 已死”这种说法并不成立。

相反，它们可能越来越重要。Computer Use 只是整个工具链中的一层，而且通常不是第一层。